

Object Oriented Programming

Java



What is Array?

A data structure which stores a fixed-size sequential collection of elements of the same type.

Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as numbers and use numbers[0], numbers[1], and ..., numbers[99] to represent individual variables.



Several Variables at Once

- It is a list of variables.
- Declare the array

```
int[] array;
```

- Initialize the array

```
array = new int[5];
```

Stores values into the array

```
array[0] = 4;
```

```
array[1] = 8;
```

```
array[2] = 7;
```

```
array[3] = 1;
```

```
array[4] = 9;
```



Two Dimensional Array

```
int[][] array = new int[2][3];
```

Stores values into the array

```
array[0][0] = 2;
```

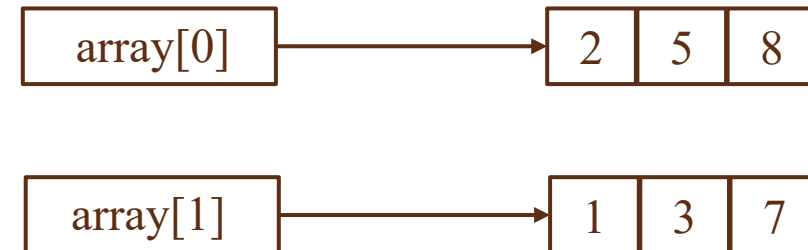
```
array[0][1] = 5;
```

```
array[0][2] = 8;
```

```
array[1][0] = 1;
```

```
array[1][1] = 3;
```

```
array[1][2] = 7;
```



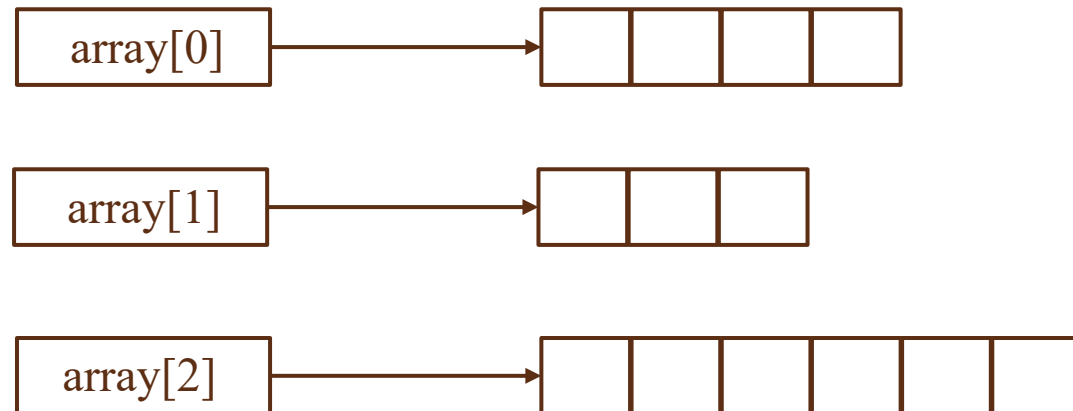
Two Dimensional Array

```
int[][] array = new int[3][];
```

```
array[0] = new int[4];
```

```
array[1] = new int[3];
```

```
array[2] = new int[6];
```



Processing Arrays

Passing Arrays to Methods

```
public void printArray(int[] array) {  
    for (int i = 0; i < array.length; i++) {  
        System.out.print(array[i] + " ");  
    }  
}
```



Processing Arrays

Returning an Array from a Method

```
public int[] reverse(int[] array) {  
    int[] result = new int[array.length];  
  
    for (int i = 0, j = result.length - 1; i < array.length; i++, j--) {  
        result[j] = array[i];  
    }  
    return result;  
}
```



ArrayList in Java

- Standard Java arrays are of a **fixed length**.
- After arrays are created, they cannot grow or shrink, which means we can not **resize** Arrays.
- Arrays are useful when we know in advance how many elements the array is to hold.

What shall we do when we do not know the size in advance?

- ArrayList is a collection which provides the implementation of resizable array.



ArrayList in Java

- ArrayList supports **dynamic arrays** that can grow as needed.
- ArrayLists are created with an initial size.
- When this size is exceeded, the collection is automatically **enlarged**.
- When objects are removed, the array may be **shrunk**.



ArrayList Constructors

The ArrayList class supports three constructors:

- ArrayList() : creates an empty array list with an initial capacity sufficient to hold 10 elements.
- ArrayList(int capacity) : creates an array list that has the specified initial capacity.
- ArrayList(Collection c) : creates an array list that is initialized with the elements of the collection c.



ArrayList Methods

Return type	Method
boolean	add(Object element)
void	add(int index, Object element)
void	clear()
boolean	contains(Object element)
Object	get(int index)
boolean	isEmpty()
Object	remove(int index)
int	size()



ArrayList Example

```
public class SampleArrayList {  
    public static void main(String[] args){  
        ArrayList<Student> arr = new ArrayList();  
        Student o1 = new Student("abc", 29);  
        Student o2 = new Student("xyz", 30);  
        Student o3 = new Student("pqr", 31);  
        arr.add(o1);  
        arr.add(o2);  
        arr.add(o3);  
    }  
}
```

```
        for(int i=0; i<arr.size();i++){  
            Student o = arr.get(i);  
            System.out.println(o.name+" "+o.age);  
        }  
    }  
}  
  
class Student{  
    String name;  
    int age;  
    Student(String n, int a){  
        name = n;  
        age = a;  
    }  
}
```



What is a String?

- String → a sequence of characters.

- Example of strings:

“A burking dog seldom bites.”

“CSE205-JAVA TECHNOLOGY”

“12345”

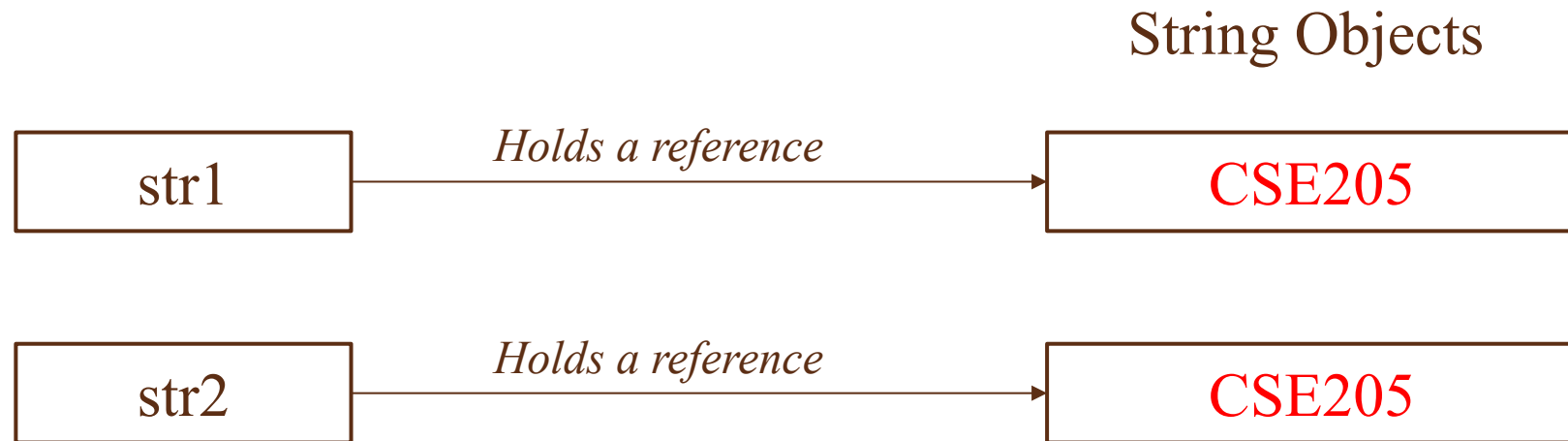
- But, unlike many other languages that implement strings as **character arrays**, java implements **strings** as **objects** of type String.
- Java handles String by two classes **String** and **StringBuffer**.



Creating String Objects

There are **2 ways** to create String objects:

1. String str1 = **new** String(**“CSE205”**);
2. String str2 = **“CSE205”**;



Understanding String Creation

```
String srt1 = "CSE205";
```

```
String srt2 = "CSE205";
```

Does Java create 2 String objects internally?

Without “new” operator for String creation:

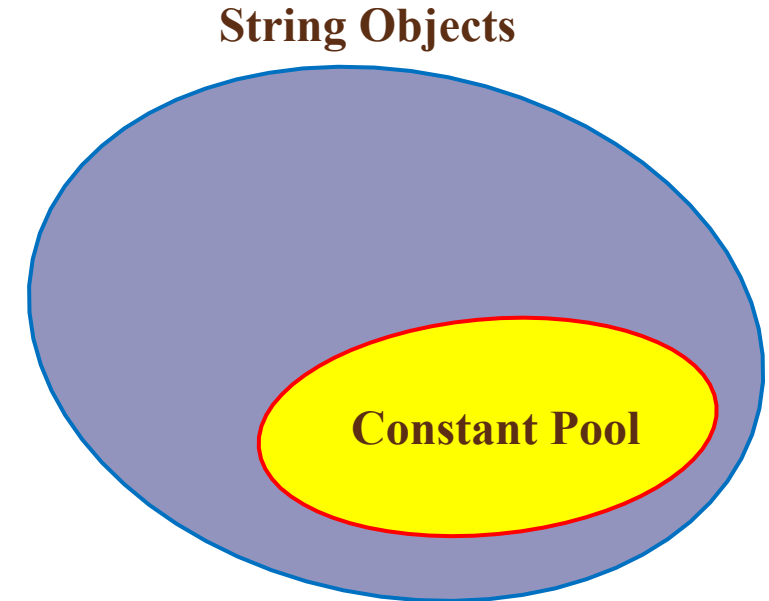
Java looks into a **String pool** (collection of String objects)

- Try to find objects with **same string value**

If **object exists** → new variable points to existing object

If **object does not exist** → new object is created

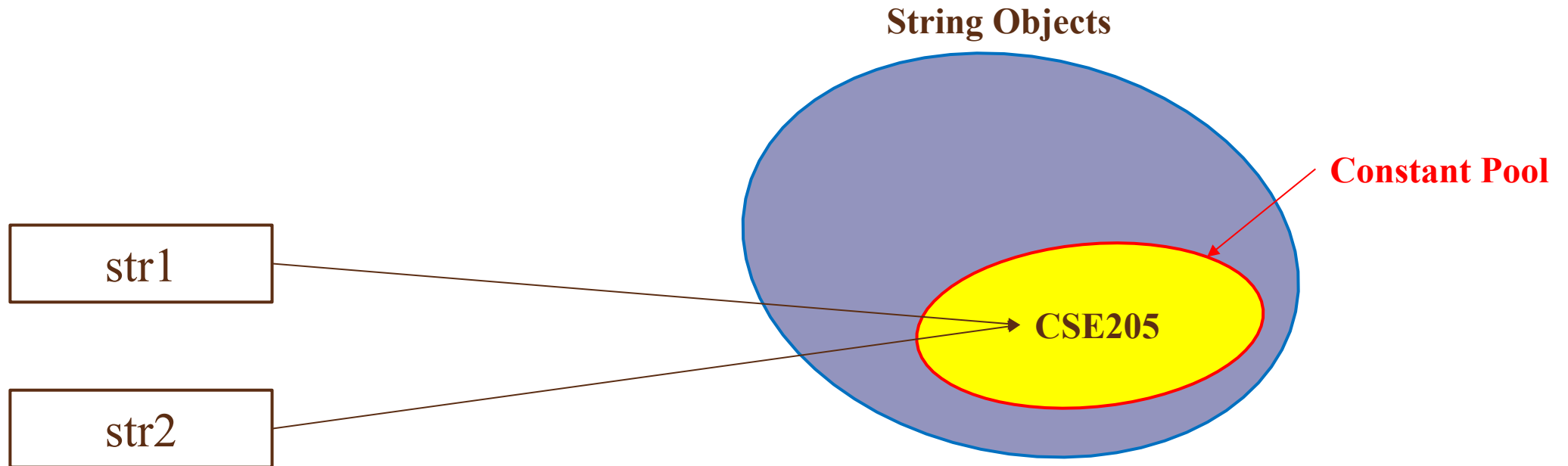
Efficiency reasons – to limit object creation



Understanding String Creation

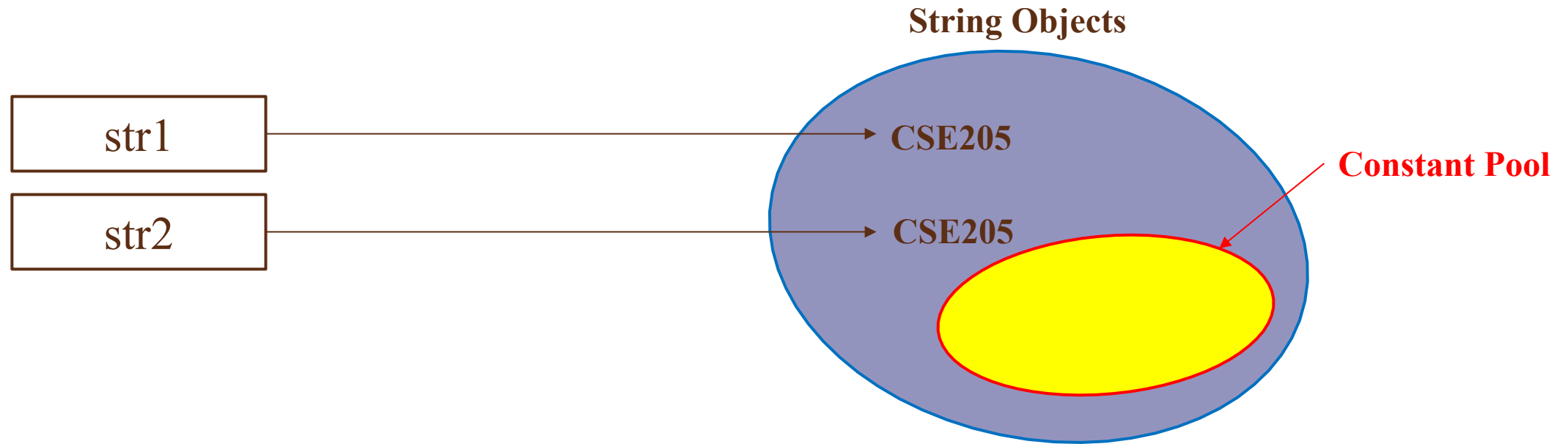
```
String str1 = "CSE205";
```

```
String str2 = "CSE205";
```



Understanding String Creation

```
String str1 = new String("CSE205");  
String str2 = new String("CSE205");
```



String Constructor

String str1= new String(); //empty object

String str2= new String("string"); //string input

String str3= new String(char[]); //char array input

String str4= new String(byte[]); //byte array input



Operations of String

- String Concatenation
- String Comparison
- Substring
- Length of String



String Concatenation

```
public static void main(String[] args) {  
    String s1 = "CSE";  
    String s2 = "205";  
    String s3 = s1+s2;  
    System.out.println(s3);  
}
```

Output: CSE205



Methods of String

Return Type	Mehtods
char	charAt(int index)
String	concat(String str)
boolean	equals(Object obj)
int	length()
String	replace(char oldChar, char newChar)
String	substring(int beginIndex, int endIndex)
String	toUpperCase()
String	toLowerCase()

